Advanced SQL Joins and Window Functions

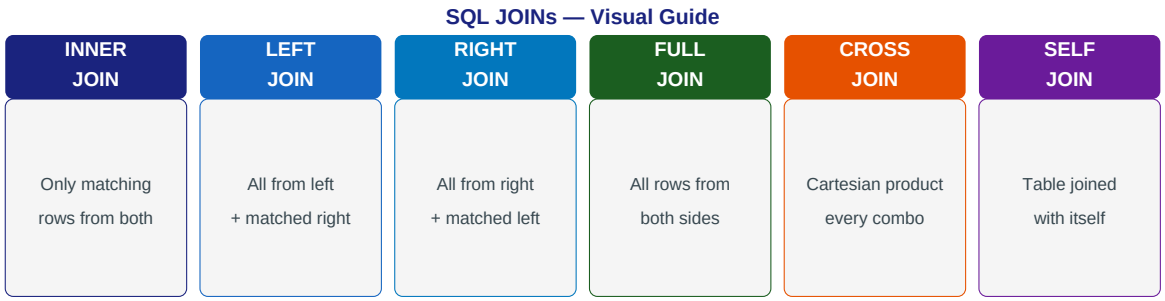
Chapter 05 - Sub-Chapter 06

SQL Databases Series

Advanced JOINs	LATERAL, SELF, FULL OUTER, anti-join
Window Functions	RANK, LAG, LEAD, running totals, moving avg
Recursive CTEs	Hierarchies, org charts, bill of materials
JSONB and Full-Text	Semi-structured data and text search in PostgreSQL

1 Advanced SQL JOINs

INNER, LEFT, SELF, LATERAL, and anti-joins



Practical JOIN patterns for e-commerce:

```
-- LEFT JOIN to find users with NO orders (anti-join pattern)
SELECT u.user_id, u.name, u.email
FROM users u
LEFT JOIN orders o ON u.user_id = o.user_id
WHERE o.order_id IS NULL;    -- NULL means no matching order row

-- SELF JOIN: find pairs of users in the same city
SELECT a.name AS user1, b.name AS user2, a.city
FROM users a JOIN users b
    ON a.city = b.city AND a.user_id < b.user_id  -- < avoids duplicates
ORDER BY a.city;

-- LATERAL JOIN: execute a subquery for each row of the left table
-- Get the last 3 orders for each user (impossible with normal JOIN)
SELECT u.name, recent.order_id, recent.total, recent.created_at
FROM users u
CROSS JOIN LATERAL (
    SELECT order_id, total, created_at
    FROM orders o
    WHERE o.user_id = u.user_id
    ORDER BY created_at DESC
    LIMIT 3
) AS recent
WHERE u.status = 'active';

-- FULL OUTER JOIN: combine two datasets, including non-matching rows
SELECT COALESCE(a.month, b.month) AS month,
       COALESCE(a.web_orders, 0)  AS web_orders,
       COALESCE(b.app_orders, 0)  AS app_orders
FROM monthly_web_stats  a
FULL OUTER JOIN monthly_app_stats b ON a.month = b.month
ORDER BY month;
```

2 Window Functions

Ranking, running totals, moving averages

Window functions perform calculations across a set of rows related to the current row without collapsing them into one (unlike GROUP BY). They are among the most powerful features in SQL — essential for analytics and reporting.

Window Functions — Aggregate Without Losing Rows

dept	name	salary	RANK()	SUM() OVER dept	LAG(salary)
Eng	Alice	90000	1	240000	—
Eng	Bob	80000	2	240000	90000
Eng	Carol	70000	3	240000	80000
Sales	Dave	60000	1	110000	—
Sales	Eve	50000	2	110000	60000

Unlike GROUP BY: each row kept — window functions add computed columns without collapsing rows

Ranking functions — RANK, DENSE_RANK, ROW_NUMBER:

```
-- Sales leaderboard: rank salespeople by revenue within each region
SELECT
    region,
    salesperson,
    revenue,
    RANK()       OVER (PARTITION BY region ORDER BY revenue DESC) AS rank,
    DENSE_RANK() OVER (PARTITION BY region ORDER BY revenue DESC) AS dense_rank,
    ROW_NUMBER() OVER (PARTITION BY region ORDER BY revenue DESC) AS row_num
FROM sales_data;
-- RANK: gaps after ties (1,1,3)
-- DENSE_RANK: no gaps (1,1,2)
-- ROW_NUMBER: always unique (1,2,3)

-- Get TOP 1 per group (the N+1 problem solved in pure SQL)
SELECT region, salesperson, revenue
FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY region ORDER BY revenue DESC) AS rn
    FROM sales_data
) ranked
WHERE rn = 1;  -- exactly one per region, no tie issues
```

Aggregate window functions — running totals and moving averages:

```
-- Running total of daily revenue (cumulative sum)
SELECT
    order_date,
    daily_revenue,
    SUM(daily_revenue) OVER (ORDER BY order_date
                             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
                            ) AS cumulative_revenue
FROM daily_sales
ORDER BY order_date;

-- 7-day moving average (useful for smoothing noisy data)
SELECT
```

```
    order_date,
    daily_revenue,
    AVG(daily_revenue) OVER (
        ORDER BY order_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  -- 7 rows including current
    ) AS moving_avg_7d
FROM daily_sales;

-- Percent of total: each order's share of user's total spending
SELECT
    user_id, order_id, total,
    ROUND(100.0 * total / SUM(total) OVER (PARTITION BY user_id), 2) AS pct_of_user_total,
    SUM(total) OVER (PARTITION BY user_id) AS user_total
FROM orders
ORDER BY user_id, total DESC;
```

LAG and LEAD — compare to previous/next row:

```
-- Month-over-month revenue growth
SELECT
    month,
    revenue,
    LAG(revenue, 1) OVER (ORDER BY month) AS prev_month_revenue,
    ROUND(100.0 * (revenue - LAG(revenue,1) OVER (ORDER BY month))
              / NULLIF(LAG(revenue,1) OVER (ORDER BY month), 0), 2) AS pct_growth,
    LEAD(revenue, 1) OVER (ORDER BY month) AS next_month_revenue
FROM monthly_revenue;

-- Detect gaps: consecutive order IDs where a gap indicates missing orders
SELECT order_id,
       LAG(order_id) OVER (ORDER BY order_id) AS prev_id,
       order_id - LAG(order_id) OVER (ORDER BY order_id) AS gap
FROM orders
HAVING order_id - LAG(order_id) OVER (ORDER BY order_id) > 1;
```

3 Recursive CTEs and Advanced Queries

Hierarchies, graphs, and complex analytics

Recursive CTEs allow you to traverse hierarchical data — organizational charts, category trees, bill of materials — entirely in SQL, without application-side loops.

```
-- Organization chart: find all reports (direct and indirect) of a manager
-- Table: employees(id, name, manager_id)

WITH RECURSIVE org_chart AS (
    -- Base case: the manager herself
    SELECT id, name, manager_id, 0 AS depth, ARRAY[id] AS path
    FROM employees
    WHERE name = 'Alice'

    UNION ALL

    -- Recursive case: find all employees who report to any employee found so far
    SELECT e.id, e.name, e.manager_id, oc.depth + 1, oc.path || e.id
    FROM employees e
    JOIN org_chart oc ON e.manager_id = oc.id
    WHERE NOT e.id = ANY(oc.path)   -- prevent infinite loops
)
SELECT id, name, depth,
       REPEAT('  ', depth) || name AS indented_name   -- visual hierarchy
FROM org_chart
ORDER BY path;

-- Bill of materials: find all components of a product, recursively
WITH RECURSIVE bom AS (
    SELECT component_id, parent_id, qty, 1 AS level
    FROM product_parts WHERE parent_id IS NULL   -- top level

    UNION ALL

    SELECT p.component_id, p.parent_id, p.qty * b.qty AS total_qty, b.level + 1
    FROM product_parts p JOIN bom b ON p.parent_id = b.component_id
)
SELECT c.name, SUM(bom.total_qty) AS total_needed
FROM bom JOIN components c ON bom.component_id = c.id
GROUP BY c.name;
```

FILTER clause — conditional aggregation:

```
-- Pivot-style report: count orders by status in one query
SELECT
    COUNT(*) FILTER (WHERE status = 'PENDING')   AS pending_count,
    COUNT(*) FILTER (WHERE status = 'CONFIRMED') AS confirmed_count,
    COUNT(*) FILTER (WHERE status = 'SHIPPED')   AS shipped_count,
    COUNT(*) FILTER (WHERE status = 'DELIVERED') AS delivered_count,
    SUM(total) FILTER (WHERE status = 'CONFIRMED') AS confirmed_revenue
FROM orders
WHERE created_at >= DATE_TRUNC('month', NOW());
-- One scan, multiple aggregations with different conditions
-- Much cleaner than multiple subqueries or CASE statements
```

4 JSONB and Full-Text Search

PostgreSQL beyond relational

PostgreSQL is not just a relational database — it has first-class support for JSON documents, full-text search, arrays, and more. This lets you use one database for both structured and semi-structured data.

JSONB — flexible schema within PostgreSQL:

```
-- Products with varied attributes stored as JSONB
CREATE TABLE products (
    product_id BIGSERIAL PRIMARY KEY,
    name       VARCHAR(200) NOT NULL,
    category   VARCHAR(50)  NOT NULL,
    attributes JSONB        NOT NULL DEFAULT '{}'
);
-- Electronics: {"cpu": "i7", "ram_gb": 16, "warranty_years": 2}
-- Clothing:    {"sizes": ["S","M","L"], "material": "cotton", "waterproof": false}
-- Food:        {"calories": 350, "allergens": ["nuts"], "organic": true}

-- Insert with JSONB
INSERT INTO products (name, category, attributes) VALUES
    ('Trail Boot', 'clothing', '{"sizes":["M","L"],"waterproof":true,"material":"leather"}'),
    ('Laptop Pro', 'electronics', '{"cpu":"i9","ram_gb":32,"storage_gb":1000}');

-- Query JSONB fields
SELECT name, attributes->>'cpu' AS cpu, (attributes->>'ram_gb')::int AS ram
FROM products WHERE category = 'electronics'
  AND (attributes->>'ram_gb')::int >= 16;

-- JSONB containment: find products that contain certain attributes
SELECT name FROM products
WHERE attributes @> '{"waterproof": true}';    -- GIN index makes this fast!

-- Array operations
SELECT name FROM products
WHERE attributes->'sizes' ? 'M';               -- has 'M' in sizes array
CREATE INDEX idx_prod_attrs ON products USING GIN (attributes);
```

Full-text search — better than LIKE for text:

```
-- Full-text search on articles
CREATE TABLE articles (
    id         BIGSERIAL PRIMARY KEY,
    title      TEXT NOT NULL,
    body       TEXT NOT NULL,
    search_vec TSVECTOR GENERATED ALWAYS AS (
        setweight(to_tsvector('english', title), 'A') ||
        setweight(to_tsvector('english', body),  'B')
    ) STORED
);
CREATE INDEX idx_articles_fts ON articles USING GIN (search_vec);

-- Search: find articles about "postgresql performance"
SELECT id, title,
       ts_rank(search_vec, query) AS rank,
       ts_headline('english', body, query,
           'MaxWords=50, MinWords=20, ShortWord=3') AS snippet
FROM articles,
     to_tsquery('english', 'postgresql & performance') AS query
```

```
WHERE search_vec @@ query
ORDER BY rank DESC
LIMIT 10;
```

Key Rules

- Use LATERAL JOIN to get top-N rows per group -- more readable than ROW_NUMBER subquery.
- Window functions: PARTITION BY = grouping, ORDER BY = sort within window, ROWS BETWEEN = frame.
- LAG/LEAD for time-series comparisons; RANK/DENSE_RANK for leaderboards.
- Recursive CTEs for hierarchy traversal -- always include a cycle prevention condition.
- Use FILTER instead of CASE WHEN for conditional aggregation -- much cleaner.
- JSONB with GIN index: semi-structured data in PostgreSQL without a separate NoSQL DB.